Roteiro de fala do seminário

Teste de aspectos e teste apoiado por aspectos - Capítulo 7 de Introdução ao Teste de Software (Lemos, Ré, Masiero, Ferrari e Cafeo). Equipe Happy Paws: Bernardo Vargens Broedel, Eduardo Capobiango Ferraz, Mateus Biancardi da Silva, Miguel Zon Murad e Victor Augusto Fernandes Rangel.

Divisão sugerida e tempo

O seminário tem de 25 a 35 minutos e todos os cinco integrantes precisam apresentar. A divisão abaixo soma 30 minutos - cerca de um minuto por slide - e pode ser trocada à vontade; o que não pode é alguém ficar de fora. Depois da apresentação o professor faz perguntas individuais, então cada um deve dominar pelo menos o seu bloco e o vocabulário da Parte 1.

Bloco	Slides	Apresenta	Tempo
Abertura	1-3	Eduardo	2 min
Parte 1 - Fundamentos de POA	4-9	Eduardo	5 min
Parte 2 - Teste apoiado por aspectos	10-16	Bernardo	6 min
Parte 3 - Testar programas OA	17-20	Mateus	4 min
Parte 4 - Teste estrutural de programas OA	21-25	Miguel	6 min
Parte 5 - Estados, mutação e ordenação	26-32	Victor	5 min
Fechamento	33-34	Eduardo	2 min
Total	1-34		30 min

Como usar: abaixo de cada slide está a nota gravada no arquivo .pptx - é o que se espera dizer naquele slide, não o que está escrito nele. No PowerPoint, as mesmas notas aparecem no modo apresentador.

Abertura · slides 1-3 · Eduardo · 2 min

Slide 1 - Teste de aspectos e teste apoiado por aspectos

Boa tarde. Nosso seminário é sobre o capítulo 7 do livro Introdução ao Teste de Software: teste de aspectos e teste apoiado por aspectos. O capítulo trata da relação entre a programação orientada a aspectos e a atividade de teste, nos dois sentidos.

Slide 2 - Duas perguntas guiam o capítulo

O capítulo é organizado em torno destas duas perguntas. Primeiro a POA como ferramenta de apoio ao teste, depois o teste de programas que usam POA. A observação do quadro é a chave: é a mesma propriedade que ajuda num caso e atrapalha no outro.

Slide 3 - Roteiro

O seminário tem cinco partes. A divisão de quem apresenta o quê é uma sugestão - ajustem como preferirem, lembrando que todos devem participar e que as perguntas do professor no fim são individuais.

Parte 1 - Fundamentos de POA · slides 4-9 · Eduardo · 5 min

Perguntas que podem cair neste bloco:

- O que é um interesse transversal? Dê um exemplo e diga por que a OO não o resolve.
- Diferença entre ponto de junção, conjunto de junção e adendo.
- O que é quantificação? E inconsciência (obliviousness)?
- O que faz uma declaração intertipos? Em que ela difere de um adendo?
- Diferença entre aspecto de desenvolvimento e aspecto de produção.

Slide 4 - Fundamentos de POA

Começamos pelos conceitos de programação orientada a aspectos, porque sem eles nada do resto faz sentido.

Slide 5 - O problema: interesses transversais

Interesses como registro, temporização, faturamento e segurança não ficam em um módulo só: eles atravessam o sistema. Os dois sintomas são espalhamento e entrelaçamento. A orientação a objetos sozinha não resolve isso.

Slide 6 - A proposta da POA

A POA separa o código-base dos aspectos e usa um combinador para juntá-los. Duas ideias centrais: quantificação, que é o aspecto alcançar muitos pontos de uma vez, e inconsciência, que é o código-base não precisar saber disso.

Slide 7 - Vocabulário essencial

Estes quatro termos aparecem o tempo todo no capítulo. Ponto de junção é onde o aspecto pode agir; conjunto de junção é a expressão que escolhe esses pontos; adendo é o que executa; e a declaração intertipos permite ao aspecto acrescentar estrutura às classes.

Slide 8 - AspectJ: o aspecto de temporização

Este é o aspecto que o capítulo usa como exemplo. Ele introduz um temporizador na classe Conexao, define um conjunto de junção para as chamadas de desconecta e, num adendo posterior, para o temporizador e acumula o tempo nos clientes. Repare que a classe Conexao não sabe nada disso.

Slide 9 - Dois propósitos para os aspectos

Guardem esta distinção: aspecto de desenvolvimento some antes da entrega; aspecto de produção fica. A primeira metade do capítulo trata dos de desenvolvimento, a segunda do teste dos de produção.

Parte 2 - Teste apoiado por aspectos · slides 10-16 · Bernardo · 6 min

Perguntas que podem cair neste bloco:

- Por que o imitador virtual é melhor que o imitador tradicional?
- Que tipo de adendo o imitador virtual usa e por quê? (around, porque substitui a execução original)
- Como o aspecto torna observável um estado interno do componente?
- Diferença entre declare error e declare warning.
- Que tipo de adendo implementa precondição, pós-condição e invariante?

Slide 10 - Teste apoiado por aspectos

Na segunda parte vemos como a POA ajuda a testar programas comuns, orientados a objetos.

Slide 11 - Por que aspectos ajudam a testar

A ideia é simples: o código que apoia o teste também atravessa o sistema. Como aspecto, ele entra e sai sem deixar rastro. Vamos ver cinco usos concretos.

Slide 12 - 1. Imitadores virtuais de objetos

O mock tradicional obriga a mexer nas classes do sistema. O imitador virtual usa um adendo around que intercepta a execução e, se houver um valor simulado registrado, devolve esse valor sem deixar o método original rodar. Todo o código de teste fica no caso de teste.

Slide 13 - 2. Teste embutido em componentes

Aqui os casos de teste viram métodos introduzidos no componente pelo aspecto, complementando a interface do componente com operações de teste. O aspecto também mantém um atributo de estado, atualizado por adendos, o que permite verificar transições que de fora seriam invisíveis.

Slide 14 - 3. Verificação de padrões de código

Dois exemplos: uma regra de projeto, que proíbe chamar um método a partir de fora de uma classe específica, e um padrão de codificação, que exige acesso a atributos por métodos set. O primeiro gera erro, o segundo um aviso, ambos em tempo de compilação.

Slide 15 - 4. Invariantes, pré e pós-condições

As três formas do projeto por contrato viram adendos: before para precondição, after returning para pós-condição e um adendo sobre a escrita do atributo para invariante. Sem POA, essas verificações ficariam espalhadas e misturadas ao código funcional.

Slide 16 - 5. Instrumentação de código

A quinta técnica é a instrumentação: usar adendos para registrar o que acontece. O exemplo monitora a escrita de um atributo. E aqui fecha a segunda parte: em todos os casos o ganho é o mesmo - código de teste separado, com alcance amplo e remoção limpa.

Parte 3 - Testar programas OA · slides 17-20 · Mateus · 4 min

Perguntas que podem cair neste bloco:

- Cite duas das quatro características que tornam o teste de programas OA diferente.
- Explique a classe 1 da taxonomia (restrição incorreta) com um exemplo.
- Por que a precedência entre aspectos pode gerar falha? (a temporização tem de vir antes do faturamento)
- Qual classe de defeito trata do cflow?

Slide 17 - Testar programas OA

Invertemos agora o ponto de vista: o aspecto deixa de ser ferramenta de teste e passa a ser o código sob teste.

Slide 18 - O que torna o teste de programas OA diferente

Quatro características. A primeira e a segunda dizem que o aspecto não existe sozinho e é acoplado ao contexto onde entra. A terceira, que o código-fonte não deixa ver as dependências. A quarta, que o comportamento é emergente: o defeito pode nascer da combinação, não de um módulo. Por isso o teste de OA precisa de modelos de defeitos próprios.

Slide 19 - Taxonomia de defeitos de Alexander et al.

Esta taxonomia é a referência do capítulo. As duas primeiras classes tratam dos conjuntos de junção e da precedência; a terceira e a quarta, da violação de contratos e invariantes pelos adendos; a quinta é específica de cflow; a sexta, das dependências de controle. Os critérios que veremos a seguir foram desenhados para revelar defeitos destas classes.

Slide 20 - Dois defeitos no exemplo da telefonia

Dois exemplos concretos na aplicação de telefonia. No primeiro, o conjunto de junção só pega a chamada explícita a desconecta. No segundo, temporização e faturamento disputam o mesmo ponto de junção e a ordem importa. Em ambos, nada no código-base denuncia o problema.

Parte 4 - Teste estrutural de programas OA · slides 21-25 · Miguel · 6 min

Perguntas que podem cair neste bloco:

- Quais são as unidades testadas em um programa OA? (método, método intertipos e adendo)
- O que é um nó transversal no grafo AODU?
- Diferença entre Todos-Nósc, Todas-Arestasc e Todos-Usosc.
- O que o grafo CoDU acrescenta ao AODU? O que é a profundidade d?
- Como o CoDU trata o adendo around e o comando proceed()?

Slide 21 - Teste estrutural de programas OA

A quarta parte adapta o teste estrutural - fluxo de controle e fluxo de dados - para programas orientados a aspectos.

Slide 22 - Do bytecode ao grafo AODU

O modelo é construído a partir do bytecode: instrução por instrução no grafo de instrução, depois agrupado no AODU com informação de definições e usos. O elemento novo é o nó transversal, que marca onde um adendo executa. E as unidades testadas passam a ser três: métodos, métodos intertipos e adendos.

Slide 23 - Critérios transversais de unidade

Os três critérios clássicos revisitados. A ideia comum é saber quais elementos cobertos são específicos do contexto OA: assim o testador sabe quando os casos de teste estão de fato ativando os aspectos. A coluna da direita mostra os requisitos reais gerados para o adendo de contorno do aspecto de transferência.

Slide 24 - Integração contextual: o grafo CoDU

O CoDU junta o AODU da unidade em teste com os AODU de tudo que executa no contexto dela, até a profundidade escolhida. Os cinco casos especiais são o que faz o modelo corresponder ao comportamento real - em especial o tratamento do around e do proceed. Os critérios são os mesmos três, agora sobre os elementos integrados, e os usos são restritos às variáveis de comunicação.

Slide 25 - JaBUTi/AJ

A JaBUTi/AJ torna os critérios utilizáveis: instrumenta o bytecode, deriva os requisitos, mostra o grafo e calcula a cobertura. O fluxo prático é olhar os requisitos não cobertos e escrever casos de teste JUnit para eles. Vale dizer que é um protótipo acadêmico.

Parte 5 - Estados, mutação e ordenação · slides 26-32 · Victor · 5 min

Perguntas que podem cair neste bloco:

- O que o ASM acrescenta ao FREE?
- Como a árvore de transições vira requisito de teste?
- Por que a cobertura multicondicional é necessária?
- Cite um operador de cada um dos três grupos e diga o que ele faz.
- O que é um mutante equivalente? Por que a Proteum/AJ consegue detectar alguns sozinha?
- Para que serve o algoritmo de Briand? Como se calcula o peso de uma aresta?
- O que é dependência bidirecional e o que fazer com ela?

Slide 26 - Estados, mutação e ordenação

A última parte cobre outras três técnicas adaptadas: teste baseado em estados, teste de mutação e a ordenação de classes e aspectos para o teste de integração.

Slide 27 - Teste baseado em estados: do FREE ao ASM

O FREE modela a classe; o ASM acrescenta a notação para os aspectos. No exemplo da conta bancária, três mudanças de requisito implementadas com aspectos criam o estado Devedora, levam ao Bloqueada quando o limite é estourado e endurecem a condição do Inativa. Tudo que é tracejado no modelo só existe por causa dos aspectos.

Slide 28 - Árvore de transições e cobertura multicondicional

Da árvore de transições saem os requisitos: cada caminho é um caso de teste. Mas as guardas compostas exigem mais: a cobertura multicondicional pede todas as combinações das condições. A tabela mostra as quatro combinações da transição desativar - só uma leva ao estado Inativa, e as outras três têm de entrar na árvore expandida.

Slide 29 - Teste de mutação para AspectJ

Os operadores se dividem em três grupos, conforme o elemento que atacam: conjuntos de junção, expressões declare e adendos. Vale notar como eles espelham a taxonomia de defeitos: PWIW e PSWR atacam a restrição incorreta; DAPC e DAPO, a precedência; ABHA e APSR, a lógica dos adendos. A ressalva do rodapé é importante numa pergunta de prova.

Slide 30 - Exemplo: o operador PWIW

Exemplo concreto: o conjunto de junção da temporização sofre a inserção de curingas. Cada par de linhas do livro é um mutante. O conceito importante aqui é o de mutante equivalente: quando a mutação não muda os pontos de junção selecionados, nenhum teste pode matá-lo, e a ferramenta reconhece isso sozinha - no exemplo do livro, 16 mutantes.

Slide 31 - Ordenação de classes e aspectos

O problema da ordem de teste vem do mundo OO e é resolvido com o ORD mais o algoritmo de Briand: Tarjan acha os ciclos, e a aresta de associação de maior peso é removida. Em POA o ORD ganha dois tipos de aresta, P para conjuntos de junção e IT para declarações intertipos, e a mesma estratégia continua valendo. A tendência é testar classes antes dos aspectos.

Slide 32 - Quando a classe conhece o aspecto

Quando a classe sabe do aspecto, a dependência vira bidirecional. O capítulo apresenta três saídas: implementação alternativa das classes, testar os aspectos primeiro, ou quebrar o aspecto em dois para desfazer o ciclo. A escolha é uma troca entre número de pseudocontrolados e retrabalho com teste de regressão.

Fechamento · slides 33-34 · Eduardo · 2 min

Slide 33 - Considerações finais

Fechando: o capítulo tratou duas perspectivas. A POA ajuda a testar programas OO separando a infraestrutura de teste do código funcional; e testar programas OA exige adaptar as técnicas existentes, com modelos, critérios, operadores e ferramentas próprios. As tendências citadas são teste baseado em conjuntos de junção, integração contextual e mutação com custo reduzido.

Slide 34 - Obrigado!

Obrigado. Estamos à disposição para as perguntas - lembrem que o professor pergunta individualmente, então cada um deve dominar pelo menos a sua parte e o vocabulário da Parte 1.